Take Home Exercise: Operations Control Room (Full Stack)

Job Profile: Full Stack Engineer

Build a small, working web app using a public mobility dataset and one live external API.

💡 **The application should tell a marketplace operator the single most important action to take right now.** It should explain why the recommendation matters, show the supporting evidence, and communicate what could make the recommendation wrong.

📊 Historical mobility data may be used to establish expected patterns or baselines. The live API should provide the current context. **The application should combine these sources to help the operator make a better-informed decision.**

🧠 A polished submission could easily require 12-20 hours, but we DO NOT expect to deliver everything. **Keep the scope to approximately 8-10 hours, prioritizing whatever YOU believe is important for a functional prototype.** Heuristics are acceptable with reasonable assumptions. We value a thoughtful product and clear trade-offs over a large number of features.

1. Technical requirements

- The backend must be written in **Go**. You may use the standard library or a lightweight Go framework (or router!).
- The application must use a simple relational database (2-3 tables) such as **SQLite** or PostgreSQL.
    - Include the database schema or migrations required to run the application.
    - At least one meaningful aggregation, comparison, or ranking must be performed in SQL rather than entirely in Go.
    - Basic, correct SQL is sufficient; thoughtful use of more advanced SQL will be a signal that you have experience with data (but this is just a nice to have!).
- Use a public mobility dataset and one live external API.
    - Suggested sources are the Chicago Transportation Network Providers trip dataset and Open-Meteo, but reasonable alternatives are acceptable.
- Build a minimal, usable frontend connected to the Go API.
    - React with TypeScript is recommended, but you may use another reasonable frontend stack.

    - Include two or three focused automated tests that cover important business logic, SQL behavior, or external API failure handling.
    - Test volume is not evaluated.

2. Product requirements

- The app should only show one current recommendation. Do not build a general-purpose dashboard or a collection of unrelated insights.
    - If no intervention is needed, clearly explain why and still provide useful supporting evidence.
- Explain what could make the recommendation incorrect, incomplete, or misleading.
    - You may communicate this in the application, the repository documentation, or both.
    - Making uncertainty clear and useful within the product may receive additional consideration.
- Make the recommendation understandable and actionable for a city operator.
    - You are free to choose the signal, heuristic, SQL approach, evidence, failure mode, and visual presentation.
    - Creativity will be assessed by the usefulness of the decision, the quality of the reasoning, and the clarity of the product (not by the number of features!)
    - Additional dashboards, machine-learning models, authentication systems, or infrastructure are not inherently better.
    - Add something only when it materially improves the core product.

3. What to submit

- An accessible GitHub repository or repository on another code-hosting platform.
    - A README with **clear local setup and run instructions**.
    - All schema definitions, migrations, dependencies, and configuration examples required to run the project.
    - Screenshots or a short demonstration of the working product.
    - A short architecture overview.
    - An explanation of the database design and important SQL queries.
    - Two or three focused automated tests.
- The assumptions, trade-offs, and known limitations of the recommendation.
- Your AI-use disclosure is described below.

❗ **The evaluator should be able to run the application locally with one or two documented commands.**
Docker is welcome but not required. Deployment is optional.

A working deployed URL may receive additional consideration because it lets the reviewer experience the web app without building it locally.
However, deployment complexity should not take priority over application quality.

4. AI Use and Follow-up Interview

AI tools are allowed and expected. Using AI will not count against you. We are interested in how you use these tools, validate their output, and make your own technical and product decisions.

Include an AI-use section in the repository describing, with as much useful detail as possible:

- Which AI tools did you use? What did you use each tool for?
    - Examples of important suggestions or outputs you accepted.
    - At least one suggestion you rejected or substantially changed, and why.
- How did you review and validate AI-generated code? Any incorrect assumptions or risks introduced by the tools.
- The parts of the solution you found most technically challenging.

The submission will be followed by an interview based on your solution. During that interview, you will be asked to:

- Demonstrate the working product.
- Explain the product decision, architecture, Go implementation, database design, and SQL.
- Discuss important trade-offs and your use of AI.
- Work through a small change, extension, or debugging scenario based on your codebase.

You may use your normal development tools, including AI, during the follow-up. The purpose is to understand how you reason, evaluate suggestions, and work with the application (not to test memorization of syntax).